

ANCHOR: Causally-Guided Agentic Debugging for Runtime Behavior Localization in Web Applications

Anonymous Author(s)

Submission to ICSE 2027 – Double-blind review.

APPENDIX A JAVASCRIPT RUNTIME AND WEB PLATFORM TERMINOLOGY

This section defines JavaScript runtime and Web platform terminology used in the paper. These definitions are intended to aid readers from general software engineering backgrounds. We follow standard definitions from the ECMAScript specification, the HTML Living Standard, and MDN Web Docs.

Bundle. A production JavaScript artifact generated by build tools such as webpack. It typically contains multiple modules merged into a single or a small set of files, often transformed through transpilation, tree shaking, minification, and code splitting, with original source structure no longer preserved.

Browser fingerprint. A browser-specific identifier computed from observable environment properties, such as user agent, canvas rendering behavior, screen resolution, and installed fonts, used to distinguish clients without explicit authentication.

Reflective dispatch. A dynamic invocation mechanism where property access or function calls are determined at runtime using computed identifiers (e.g., `obj[name]` or dynamically generated code).

Runtime code generation. Execution of code generated during program execution, typically via `eval`, `Function` constructor, or dynamically injected scripts.

Bundler-induced module indirection. A transformation introduced by JavaScript bundlers where module boundaries are replaced with runtime loader indirections (e.g., numeric module identifiers), obscuring static structure.

Minification. A code transformation that reduces file size by removing whitespace and comments and shortening identifiers without changing program semantics.

Control-flow flattening. An obfuscation technique that rewrites structured control flow into a dispatcher-based loop controlled by a state variable and switch statement.

String-array indirection. An obfuscation pattern where string literals are stored in arrays and accessed via computed indices.

Framework callbacks. Callbacks invoked by Web frame-

works in response to lifecycle or rendering events (e.g., React render, Vue watchers, Angular lifecycle hooks).

Microtask. An asynchronous task scheduled to execute after synchronous code completes but before the next event-loop task.

Macrotask. A task scheduled in the browser event loop queue, such as timers, I/O events, or user interactions.

Promise continuation. A callback registered via `then`, `catch`, or `finally` executed when a Promise settles.

Interceptor. A middleware-like mechanism that intercepts and possibly modifies requests or responses in a pipeline.

Sink. A program point where externally observable behavior occurs (e.g., network request, DOM update, storage write).

Chrome DevTools Protocol. A remote debugging protocol enabling programmatic interaction with Chrome for inspection and instrumentation.

Event-loop turn. One iteration of the JavaScript event loop consisting of one task execution followed by microtask flushing.

APPENDIX B MEASUREMENT PROTOCOL FOR QUANTIFYING RUNTIME SEMANTIC EXPLOSION

A. Subjects

We conducted the measurement on 14 production-grade Web applications selected from our real-world benchmark suite. Each subject contains one target behavior with an externally observable sink, including login authentication, request signing, encrypted parameter generation, and local state encoding. For each subject, we defined one scripted user-level interaction that triggers the target behavior exactly once. Table I lists all subjects.

B. Function Invocation Counting

For each subject, we instrumented all executable user-space JavaScript functions, including named functions, anonymous closures, arrow functions, and dynamically generated functions, while excluding browser-native APIs and engine-level built-ins.

TABLE I
PRODUCTION WEBSITES USED IN PRELIMINARY MEASUREMENT.

ID	Domain	Target Behavior
S1	tzxm.jxzwfw.gov.cn	request signing
S2	yngwypt.zmnyjk.com	password encryption
S3	d.weidian.com	request transformation
S4	iquicker.com.cn	login signing
S5	37.com	login signing
S6	passport.fang.com	password signing
S7	web.ewt360.com	login signing
S8	fuwu.nhsa.gov.cn	encrypted payload generation
S9	learn.open.com.cn	request signing
S10	dict.cnki.net	request transformation
S11	appmiu.com	local state encoding
S12	max.pedata.cn	request signing
S13	u.caixin.com	login signing
S14	passport.yhd.com	login signing

During one interaction, every function entry emits one runtime probe event. Let $E = \{e_1, \dots, e_n\}$ denote the ordered sequence of observed function-entry events. We count invocation events rather than distinct functions, meaning repeated calls to the same function are counted multiple times. The total number of invocation events is therefore $N_{\text{inv}} = |E|$.

Across all 14 subjects, a single interaction triggers a median of 493.5 function invocation events.

C. Asynchronous Turn Counting

To quantify asynchronous execution, we partition the invocation sequence into asynchronous turns. An asynchronous turn is defined as a maximal contiguous subsequence of invocation events executed under the same event-loop continuation. A new turn begins whenever control returns to the event loop and later resumes through a macro-task or micro-task callback, such as Promise reactions, `setTimeout`, or network-response handlers.

Let T_{async} denote the number of asynchronous turns within one interaction. Across all subjects, the median number of asynchronous turns is 7.

D. Semantic Relevance Annotation

For each target behavior t , we identify the subset of invocation events that are semantically relevant to that behavior, denoted by $E_{\text{sem}}(t)$.

An invocation event is considered semantically relevant if its executed function satisfies three criteria: (1) it is causally reachable from the user-event entry point to the observable sink; (2) it performs target-specific logic rather than generic reusable computation; and (3) it directly constructs, transforms, binds, or writes the target value. Generic utilities, framework dispatchers, event plumbing, and reusable cryptographic primitives are excluded unless they implement target-specific logic.

We define the semantic relevance ratio as $\sigma(t) = |E_{\text{sem}}(t)|/|E|$, i.e., the fraction of invocation events relevant to target behavior t . Semantic relevance is determined using

benchmark oracle annotations together with static call-graph constraints.

Across all subjects, fewer than 4.2% of invocation events are semantically relevant to any particular target behavior.

E. Summary

These measurements reveal a severe signal-to-noise imbalance in runtime execution: a single user interaction typically triggers hundreds of function invocations across multiple asynchronous stages, while only a small fraction contributes to the target behavior of interest. This quantitatively characterizes *runtime semantic explosion* and motivates behavior-aware localization instead of exhaustive runtime tracing.

APPENDIX C MOTIVATING EXAMPLE: FULL FAILURE-MODE WALKTHROUGH

This section provides a step-by-step walkthrough of the argument behind each failure mode, for readers who want the detailed reasoning underlying the condensed motivating example.

a) Structural uncertainty: static search cannot pinpoint the anchor: A natural first attempt is to inspect the JavaScript code statically by searching for `order_sig_v2`, `digest`, `hash`, `checkout`, or `fetch`. This strategy fails along three independent axes. First, the final field name appears only near the observable sink and the final dynamic write; the upstream functions that construct the signature material use neutral names such as `deriveStableFrame`, `foldEnvelopeFrame`, and `commitFrame`. Second, the true call path is partially hidden behind dynamic dispatch: the request is processed by an interceptor pipeline whose relevant stage is resolved through a runtime stage key and a registry lookup, so a static call graph may miss the true edge or conservatively include many unrelated handlers. Third, the program contains many statically similar decoy functions (`analytics_sig`, `coupon_digest`, `view_cache_key`, a retry token, an A/B bucket digest) that normalize objects, fold strings, and attach tokens just like the real path. A static analysis broad enough to reach the true sealing interceptor inevitably also reaches dozens of unrelated digest-like functions, and identifier-based ranking cannot disambiguate because the anchor and the decoys carry equally plausible names.

b) Semantic explosion: dynamic tracing drowns the anchor in noise: A single click on `SUBMIT ORDER` triggers form validation, UI lifecycle updates, analytics logging, coupon preview recomputation, cart rendering, payment eligibility checks, retry-metadata preparation, beacon construction, and many Promise/microtask callbacks. Several produce intermediate values that look relevant (`trace_id`, `preview_hash`, `coupon_digest`, `render_nonce`, `analytics_sig`, `retry_fingerprint`). A trace that records every call and mutation may contain the answer, but the relevant fragments are sparse and deeply buried, and sorting by string similarity to the target field name fails because the upstream

material is built by functions whose code never mentions `order_sig_v2`. Pure dynamic tracing therefore shifts the burden from missing information to excessive information.

c) *Causal fragmentation: the anchor is hidden across asynchronous boundaries*: The example deliberately fragments value provenance across turns: the browser profile receipt is collected through asynchronous device-info callbacks, the interceptor pipeline is driven by a `Promise.resolve`-scheduled microtask chain, and the final network call runs in a later macrotask. The synchronous stack at the sink contains only the network wrapper and a few framework adapters; the debugger’s asynchronous stack trace identifies which microtask scheduled the network call but not which function constructed the value in `body.order_sig_v2`. Recovering the anchor requires stitching synchronous stacks, asynchronous call provenance, and cross-turn value flows into one structure rather than reading any single frame.

d) *Why an LLM agent without static–dynamic coordination falls short*: An agent that begins from keyword search wades through decoy digest paths surfaced by structural uncertainty and, latching onto the most prominent digest, recovers only a generic SHA-256 utility whose body cannot explain why `body.order_sig_v2` in particular is produced. An agent that follows traces uniformly must inspect thousands of framework callbacks before reaching the interceptor. An agent that starts from the network sink sees the body only after the signature is inserted and finds no target-specific function on the synchronous stack. These failures arise not from weak reasoning but because no single source of evidence suffices and the agent has no mechanism to make the three constrain each other.

APPENDIX D

PROBLEM DEFINITION: WORKED CASES

a) *Three nested cases: Case 1 (anchor sandwiched between generic dispatch and a generic utility)*: in a chain $A \rightarrow B \rightarrow C \rightarrow D$ where A is a click handler, B a generic interceptor-pipeline dispatcher, C a checkout-specific sealing interceptor that assigns `body.order_sig_v2`, and D a reusable SHA-256 routine, the anchor is C : A and B route any request, while D is a generic hash whose code cannot by itself explain why this particular field is produced. *Case 2 (anchor above the literal binding site)*: if A is a generic submit handler that writes whatever its callee returns, B is dedicated to computing `search_sig`, and C is the underlying hash, the anchor is B even though the literal assignment occurs in A , because B is the highest chain function whose body is dedicated to the target. *Case 3 (anchor embedded in a mixed-purpose function)*: when the target value is produced by an inline statement such as `ctx.body[targetField] = digest(...)` inside a function whose remaining body handles unrelated request types, the enclosing function is still target-specific and the anchor rule selects it.

b) *Existence of f^* and the no-anchor case*: The anchor rule presupposes that at least one target-specific function exists

on the chain. Behaviors realized entirely by composition of generic utilities (for example, a request body produced by a stock serializer with no target-specific glue) admit no anchor under the rule. Our benchmark protocol excludes such tasks during construction: a candidate behavior is admitted only if expert annotators agree that at least one chain function passes the deletion-based anchor test. Methods evaluated on this benchmark may therefore assume the existence of f^* and are not required to abstain. Extending the framework to abstain on no-anchor behaviors is left to future work.

APPENDIX E

APPROACH DETAILS

This appendix expands the condensed cost model of information-gain breakpoint selection and the value-match predicate of async causal stitching.

a) *Cost model $c(b_i)$* : We instantiate $c(b_i)$ as the expected number of times b_i fires within a single task trigger, capped at a per-category constant $c_{\max}$. Breakpoints inside one-shot event handlers receive $c(b_i) = 1$. Breakpoints inside Promise continuations or microtask callbacks receive $c(b_i)$ equal to the static fan-in of the enclosing dispatch site, also capped at $c_{\max}$. Breakpoints inside rendering, polling, or interval callbacks are clipped at $c_{\max}$ a priori, since their hit count is governed by viewport and timer state rather than by the task trigger. The numerical value of $c_{\max}$ is fixed per category; the loop’s behavior is insensitive to its exact value provided that recurrent callbacks are penalized strictly more than one-shot handlers.

b) *Value match for data edges*: For *long strings and byte arrays* (length $\geq \ell_{\text{long}}$), match is exact equality on canonical bytes, hashed and indexed to allow $O(1)$ pairwise lookup. For *structured objects*, match requires identical key schemas and a key-sorted shallow value hash. For *short primitives* such as booleans, small integers, and short strings, match is equality discounted by the global occurrence frequency of the value across all collected hits, suppressing edges induced by ubiquitous values such as `true` or empty strings. For *containment relationships*, where one value is a substring or subarray of another, a partial data edge is added with a lower weight $w_{\text{partial}} \in (0, 1)$, capturing the common case of intermediate digests being concatenated into a final signature. This criterion is conservative enough that random framework values do not produce data edges, yet permissive enough that the closure-held session pepper observed at one hit is linked to the digest material observed at a later hit in a different microtask.

c) *Per-stage computational cost*: The per-iteration cost bound decomposes as follows. The breakpoint proposer issues one LLM call per iteration to enumerate candidate breakpoints with predicted outcomes. The subsequent Bayesian update evaluates the LLM likelihood $P(o_{i,j} \mid f, b_i)$ as a binary consistency judgment over the top- k slice of H_{t-1} , at most $k \cdot M$ batched judgments per iteration, batched into one LLM call per breakpoint. Causal stitching incurs one LLM classifier call per newly visited node in the backward BFS, bounded by the causal-graph depth along reachable paths from the sink.

Algorithm 1 INFOGAINPROBE: information-gain breakpoint selection

Require: hypothesis H_t , causal graph G_t

Ensure: selected breakpoint b^*

```

1:  $B_{\text{cand}}, \{(o_{i,j}, q_{i,j})\} \leftarrow \text{LLMPROPOSE}(H_t, G_t)$ 
2: for all  $b_i \in B_{\text{cand}}$  do
3:    $\text{IG}_i \leftarrow \mathcal{H}(H_t)$ 
4:   for all outcome  $o_{i,j}$  of  $b_i$  do
5:      $H' \leftarrow \text{BAYESUPDATE}(H_t, b_i = o_{i,j})$ 
6:      $\text{IG}_i \leftarrow \text{IG}_i - q_{i,j} \cdot \mathcal{H}(H')$ 
7:   end for
8:    $\text{IG}_i \leftarrow \text{IG}_i / (c(b_i) + \epsilon)$ 
9: end for
10:
11: return  $b^* \leftarrow \arg \max_{b_i} \text{IG}_i$ 
```

TABLE II
IMPLEMENTATION ENVIRONMENT.

Component	Version
Node.js	20.19.0
Chrome DevTools Protocol	Chrome 138.x
Acorn	8.16.0
Astring	1.9.0
chrome-remote-interface	0.33.3
ws	8.21.0

CDP traffic is dominated by the at most $k \leq 3$ breakpoints set per iteration plus a constant per-hit cost for capturing the 4-tuple (S_h, A_h, V_h, T_h) .

d) *Pseudocode:* Algorithms 1 and 2 give the information-gain breakpoint selector and the graph-stitching and anchor-recovery procedure, respectively; they provide a self-contained specification of these two stages of the co-refinement loop for implementers.

APPENDIX F HYPERPARAMETERS AND CLOSED FORMS

This appendix gives the implementation environment and the closed forms of the method. All values are implementation defaults; no grid search or held-out calibration is performed, and every value is fixed across datasets.

a) *LLM configuration:* The system uses an OpenAI-compatible endpoint with a unified decoding configuration: model DeepSeek-V4-Pro, temperature 0, JSON-mode responses (auto fallback on failure), and a retry budget of 3 for JSON decoding.

b) *Likelihood factorization:* For each candidate function $f \in F_C$, the system defines a multiplicative likelihood

$$L_t(f) = \text{clip}\left(L_{\text{val}}(f) \cdot L_{\text{anchor}}(f) \cdot L_{\text{pred}}(f), L_{\min}, L_{\max}\right), \quad (1)$$

Algorithm 2 STITCHCAUSALGRAPH and Anchor Recovery

Require: previous graph G_{t-1} ; new hits $\{h_1, \dots, h_n\}$; sink node n_{sink}

Ensure: updated graph G_t ; anchor candidate set A_t

```

1:  $G_t \leftarrow G_{t-1}$ 
2: for all hit  $h$  in  $\{h_1, \dots, h_n\}$  do
3:    $(S_h, A_h, V_h, T_h) \leftarrow \text{CAPTURE}(h)$ 
4:   add nodes and sync edges from  $S_h$  to  $G_t$ 
5:   add async edges from  $A_h$  to  $G_t$ 
6:   for all prior hit  $h'$  with  $T_{h'} \neq T_h$  do
7:     for all  $v_i \in V_h, v_j \in V_{h'}$  do
8:       if  $\text{VALUEMATCH}(v_i, v_j)$  then
9:         add data edge from  $h$  to  $h'$  to  $G_t$ 
10:      end if
11:    end for
12:  end for
13: end for
14:  $A_t \leftarrow \emptyset$ ;  $Q \leftarrow \{n_{\text{sink}}\}$ ;  $\text{VISITED} \leftarrow \{n_{\text{sink}}\}$ 
15: while  $Q \neq \emptyset$  do
16:    $n \leftarrow \text{DEQUEUE}(Q)$ 
17:   if  $\text{ISTARGETSPECIFIC}(n, f, d)$  and
       $\text{REACHES TRIGGER}(n, G_t)$  then
18:      $A_t \leftarrow A_t \cup \{n\}$  {do not expand predecessors of  $n$ }
19:   else
20:     for all predecessor  $n'$  of  $n$  in  $G_t$  do
21:       if  $n' \notin \text{VISITED}$  then
22:         enqueue  $n'$  into  $Q$ ;  $\text{VISITED} \leftarrow \text{VISITED} \cup \{n'\}$ 
23:       end if
24:     end for
25:   end if
26: end while
27: return  $G_t, A_t$ 
```

with global clipping bounds $L_{\min}, L_{\max}$. Let f_{hit} denote the function associated with the observed breakpoint:

$$L_{\text{val}}(f) = \begin{cases} L_{\text{val}}^+, & f = f_{\text{hit}} \wedge \text{valueMatch} \\ L_{\text{val}}^-, & f = f_{\text{hit}} \wedge \neg \text{valueMatch} \\ 1, & \text{otherwise.} \end{cases} \quad (2)$$

Let f^* be the anchor candidate and $s(f) \in [0, 1]$ its LLM score:

$$L_{\text{anchor}}(f) = \begin{cases} b_{\text{anchor}} + s(f^*), & f = f^* \\ L_{\text{reject}}, & f \neq f^*, s(f) < \theta_{\text{anchor}} \\ 0.5, & f \neq f^*, s(f) \geq \theta_{\text{anchor}} \\ 1, & \text{otherwise.} \end{cases} \quad (3)$$

$L_{\text{pred}}(f)$ is produced by the breakpoint-selection module; if unavailable, $L_{\text{pred}}(f) = 1$.

c) *Bayesian update:*

$$\tilde{H}_{t+1}(f) = H_t(f) \cdot L_t(f), \quad H_{t+1}(f) = \frac{\tilde{H}_{t+1}(f)}{\sum_{g \in F_C} \tilde{H}_{t+1}(g)}. \quad (4)$$

TABLE III
DEFAULT HYPERPARAMETERS, FIXED ACROSS DATASETS.

Symbol	Value	Description
temperature	0	LLM decoding temperature
K	3	Breakpoint-selection focus size
$T_{\max}$	20	Maximum iterations
$L_{\min}$	0.1	Likelihood lower bound
$L_{\max}$	10	Likelihood upper bound
L_{val}^+	2.5	Value-match reward
L_{val}^-	0.4	Value-mismatch penalty
b_{anchor}	8.0	Anchor base boost
L_{reject}	0.1	Anchor rejection penalty
θ_{conf}	0.9	Convergence confidence threshold
θ_{anchor}	0.7	Anchor acceptance threshold
ε	10^{-6}	IG numerical stability
τ	1.0	Structural prior temperature
w_{ast}	1	AST weight
w_{api}	0.8	API weight
w_{ent}	1	Entropy weight
w_{sink}	0.5	Sink proximity weight
D	7	Graph traversal depth
q_{p_0}	0.7	Structural prior percentile cutoff

If the denominator is zero, a uniform fallback distribution is used.

d) Redundancy and convergence: With $\hat{f} = \arg \max_f H_t(f)$,

$$\text{Redundant}(\hat{f}) = \text{LLMValid}(\hat{f}) \wedge (\text{P0Aux}(\hat{f}) \vee \text{CausalAux}(\hat{f})), \quad (5)$$

$$\text{Converged} = (H_t(\hat{f}) \geq \theta_{\text{conf}}) \wedge \text{Redundant}(\hat{f}). \quad (6)$$

e) Structural prior: The structural score combines four components,

$$S(f) = w_{\text{ast}} S_{\text{ast}}(f) + w_{\text{api}} S_{\text{api}}(f) + w_{\text{ent}} S_{\text{ent}}(f) + w_{\text{sink}} S_{\text{sink}}(f), \quad (7)$$

where S_{ast} encodes AST template similarity, S_{api} API-signature relevance, S_{ent} entropy-related behavior density, and S_{sink} static proximity to sink nodes. The prior is a temperature-scaled softmax,

$$p_0(f) = \frac{\exp((S(f) - \max_g S(g))/\tau)}{\sum_{g \in F_C} \exp((S(g) - \max_h S(h))/\tau)}, \quad (8)$$

and $H_0(f) = p_0(f)$. The sink-proximity term uses the static call graph G and sink set $\mathcal{S}$: $S_{\text{sink}}(f) = 1/(d_G(f, \mathcal{S}) + 1)$, and 0 if unreachable.

f) Breakpoint-selection acquisition objective: $IG(b) = H(H_t) - \mathbb{E}_o[H(H_t | o)]$ and $\text{score}(b) = IG(b)/(c(b) + \varepsilon)$.

APPENDIX G BENCHMARK CATEGORY EXAMPLES

Table IV lists, for each of the five ANCHORBENCH behavior categories, the target observable and a representative maintainer-verified anchor named by its captured-bundle identifier.

APPENDIX H BENCHMARK CONSTRUCTION DETAILS

This appendix expands the corpus, oracle, and annotation description of ANCHORBENCH; these details are provided to support reproduction.

a) Corpus and packaging: Each case is bundled with rollup and hardened with javascript-obfuscator under a fixed seed (local-identifier mangling, string-array extraction, control-flow flattening; source maps off). We deliberately keep module-scope function names (`renameGlobals` off), modeling builds that harden locals, strings, and control flow but retain top-level names, so we can test whether the structural prior helps even when lexical base-lines still have a name signal; identifier erasure is studied separately at obfuscation-ladder tier T1. Each case splits into `agent_visible/` (the `task.json` and a DevTools capture of exactly the main-context scripts a debugger exposes at the trigger, i.e. the corpus C) and a never-visible `agent_hidden/` (un-obfuscated `src/`, `build/grading` scripts, oracle).

b) Oracle and grading: Grading is strictly top-1: a returned complete function is matched to a role entry by body SHA-256 (and a whitespace-normalized variant) with a `{file, offset}/span-containment` fallback; S_d is the matched score, and an exact anchor match counts toward strict accuracy. Because the oracle defines the target, it is established *entirely by human annotation*: annotators may consult an instrumented trace as evidence, but all chain, role, and anchor judgments are human, with the deletion-based anchor criterion used as a written guideline rather than a decidable procedure. Each role assignment is recorded with a written rationale in the released oracle, and behaviors for which no chain function satisfies the deletion-based criterion are excluded during construction. A build-time verifier re-triggers each case to confirm the observable reproduces and re-resolves every oracle span against the captured bundle by offset and hash. On a random double-annotated subset, Krippendorff’s α is 0.83 on the anchor f^* and 0.70 on the finer partial-credit roles (nominal coding); a full multi-annotator study crossed by category is left to a future revision of the benchmark.

c) Statistics: The candidate universe ranges from a minimum of 462 functions to 20,801 (median 1,735), bundles have median size 2.4 MB, the synchronous call depth from trigger to sink ranges 10–52 (median 38), and each task crosses 3–8 asynchronous turns (median 6).

d) Baseline configurations: The capability tiers A_m used in our evaluation are *static* (inspection of C only), *exec-aware* (adds console/network reads), *instrumented-exec* (adds tracing), and *debugger* (adds breakpoints, stack inspection, and variable reads); a machine-checked gate rejects any over-tier run. Within the baseline families we compare against: *Direct-LLM* is a two-pass shortlist-then-select static pipeline; *JS-DynSlice* is backward dynamic slicing, which cannot cross opaque transforms such as `SubtleCrypto`; *SWE-agent* and *LocAgent-JS* operate over read-only minified source; and *Debugger-Agent* is a matched-capability control on ANCHOR’s

TABLE IV
THE FIVE BEHAVIOR CATEGORIES OF ANCHORBENCH, WITH A REPRESENTATIVE ANCHOR NAMED BY ITS CAPTURED-BUNDLE IDENTIFIER.

Category	Target observable	Representative anchor (role)
Request signing & token derivation	proof/seal token from fields + runtime context (e.g. <code>ap_</code> -prefixed <code>account_proof</code>)	<code>encodeAccountProofEnvelope</code> : folds tuple/config/runtime state into the token, above the hash it calls
State encoding	compact stable encoding of form + UI/route state (e.g. <code>state_code</code>)	<code>encodeStateEnvelope</code> : selects the reducer and formats the code
Byte-array transformation	byte payload materialized then serialized (e.g. <code>Uint8Array</code> $\rightarrow$ <code>base64url</code> <code>byte_payload</code>)	<code>encodeByteArrayEnvelope</code> : materializes and encodes the payload
Browser fingerprinting	compact digest of navigator/screen/timezone/media/canvas (e.g. 12-char <code>browser_fingerprint</code>)	<code>assembleBrowserFingerprint</code> : selects surfaces and formats the digest
Request transformation	deterministic value written into an outgoing field (e.g. <code>X-Transform-Key</code>)	<code>buildRequestTransform</code> : packages the value before the network wrapper

exact CDP surface but without causally guided search (in the spirit of generic debugger drivers such as ChatDBG and the Chrome DevTools MCP); its full configuration, prompt, retry policy, and a failure analysis attributing its low score to a genuine capability ceiling under matched conditions are in Appendix I. The diagnostic lower bounds (BM25-Static, SimpleSink, Exec-LLM, Uniform-Tracer, Uniform-Random) isolate single capabilities. No baseline consumes any ANCHOR-specific intermediate or the oracle, and all LLM methods share backbone, decoding, and budget.

APPENDIX I DEBUGGER-AGENT MATCHED CONTROL: CONFIGURATION AND FAILURE ANALYSIS

This appendix documents the full configuration of the *Debugger-Agent* matched-capability control and analyzes the source of its low score, establishing that the score reflects a genuine capability ceiling under conditions matched to ANCHOR. *Debugger-Agent* is a ReAct/CodeAct-style agent driving a generic Chrome DevTools Protocol debugger; it shares ANCHOR’s backbone, decoding, per-task budget, anchor definition, and oracle, and differs only in lacking ANCHOR’s strategy operators (the structural prior, information-gain breakpoint selection, async causal stitching, and the co-refinement loop).

a) Budget: Table V lists the per-task budget, matched to ANCHOR’s loop: the same backbone and decoding, 3 repetitions, and a 20-step ceiling equal to ANCHOR’s $T_{\max} = 20$, under a shared token and wall-clock cap.

b) Tool surface: Over CDP the control exposes seven generic tools: `list_candidates`, `read_source`, `set_breakpoint` (0-based offset with an optional JS condition), `trigger` (replays the task interaction once, returning ordered breakpoint hits with their synchronous/asynchronous stacks and shallow locals), `read_vars` (local and closure variables at a chosen hit), `observe` (console/network/DOM), and `submit`. This is exactly ANCHOR’s CDP surface without the strategy operators.

TABLE V
Debugger-Agent MATCHED-CONTROL PER-TASK BUDGET. BACKBONE, DECODING, AND REPETITIONS FOLLOW THE SHARED EXPERIMENTAL PROTOCOL.

Knob	Value
Paradigm	ReAct + CodeAct
Backbone & decoding	shared experimental protocol
Repetitions	3
Max ReAct steps	20
Breakpoints per replay / total	6 / 24
Token budget per task	7.5×10^5
Max tokens per LLM call	1500
Wall-clock per task	600s
Source access	read-only

c) Prompt: The system prompt fixes the same anchor definition used throughout this paper, verbatim:

You are a debugging agent locating the single behaviour-anchor function in a deployed, obfuscated browser bundle. You may set breakpoints, replay the interaction, and inspect paused sync/async stacks and local/closure variables. The anchor is the first target-specific value-construction function on the dynamic chain (not routing, input collection, a wrapper, the sink, or a generic utility). Reason step by step (ReAct) and, when confident, call submit with the complete enclosing function’s file and offsets.

The user turn supplies only the task question, the observable, the interaction, and the bundle file list. The control therefore receives ANCHOR’s exact target definition, not a vaguer or misleading one.

d) Retry policy: The LLM client uses no retry or backoff: a non-200 response is thrown and recorded, and the wrapper takes top-1 as `RESOLVE(submitted answer)`, or `RESOLVE(best guess)` when nothing was submitted, where the best guess is the last function opened by `read_source`; a BM25 recall tail with top-1 abstention is the final fallback. Across the RQ1 batch this fallback never fired (no abstentions, no API errors, all runs complete), so the low score reflects genuine localization failures rather than abstention, crashes, or missing

retries.

e) *Root cause: wrong observation selection, not evidence misreading:* Of the two candidate explanations, the evidence points squarely to *breakpoint/observation selection error*. Across the RQ1 batch (70 tasks $\times$ 3 repetitions = 210 runs) the control never calls `submit` (0/70; all 210 runs end unsubmitted), so scoring falls on its best guess. It does exercise the debugger, replaying `trigger` on 47/70 tasks and setting breakpoints on 49/70, yet stays far under budget (mean 5.1/24 breakpoints) and, decisively, reads paused variables (`read_vars`) on only 7/70 tasks, reverting to static `read_source` on 46/70. Run-level it reaches the anchor f^* on just 4/210 runs (folded strict accuracy 0.00); its best guess usually lands on an off-anchor role (wrapper, sink, or generic utility), accumulating partial credit to a weighted ≈ 0.096 . The control’s own ReAct traces expose the mechanism (`case009_request_transformation`): “the breakpoints didn’t hit, so those functions aren’t on the dynamic chain for this interaction; let me look at more candidate functions.” The loop is: breakpoints placed off the dynamic value-construction chain miss on replay, no paused frame is captured, the agent abandons dynamic evidence and reverts to reading source, and the step budget expires on a static guess. The failure is thus located at *which observations to make*: the control cannot place breakpoints on the value-construction chain in an obfuscated bundle, so in most runs there is no captured runtime evidence to misjudge (`read_vars` used on only 7/70). This rules out evidence-interpretation error as the primary cause.

f) *Validity of the matched control:* The control is held to the same backbone, per-task budget, full CDP tool surface, and anchor definition as ANCHOR, and is scored without additional penalty (the BM25 abstention fallback never fired). Under these matched conditions a standard ReAct+CodeAct debugger executes dynamic debugging but does not select effective observation points on obfuscated JavaScript: 0/70 submissions, `read_vars` on only 7/70 tasks, and breakpoints that frequently miss and force a static fallback. Its ≈ 0.096 therefore reflects the capability ceiling of generic debugger access under matched conditions, and quantifies the gap closed by ANCHOR’s structural prior, information-gain breakpoint selection, and async causal stitching.

APPENDIX J

SUPPLEMENTARY FIGURES AND TABLES

This appendix collects secondary diagnostics that complement the main experimental results. Table VI gives the per-dimension structural-prior contributions (RQ2), and Figure 1 shows the iteration-count distribution of successful anchor recovery under the RQ3 component ablations; the headline cost–accuracy trade-off is reported separately with the main results.

TABLE VI
STRUCTURAL-PRIOR DIMENSION CONTRIBUTION UNDER LEAVE-ONE-OUT ABLATION (RQ2): EACH DIMENSION’S SHARE OF THE COMBINED SCORE, MEAN RECIPROCAL RANK (MRR), LOG-/LINEAR-PROBABILITY SHIFTS, AND THE HELP/HURT TASK SPLIT (H/H).

Dim	Share	MRR	$\Delta \log p$	Δp	H/H
API	50.6%	0.111	+0.078	+0.003	114/2
AST	12.9%	0.109	+0.044	+0.004	80/32
ENT	11.0%	0.117	+0.038	+0.002	91/25
Sink	25.4%	0.097	+0.018	+0.001	111/5

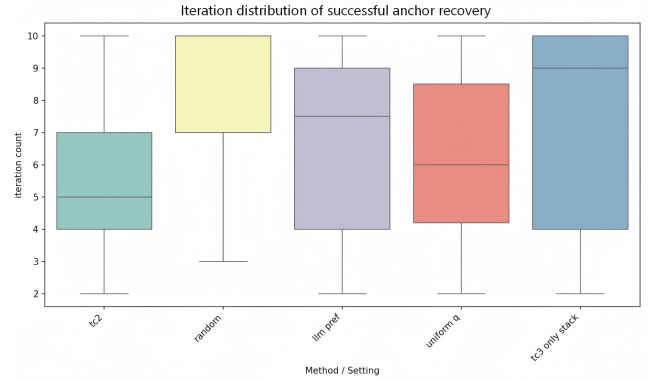


Fig. 1. Iteration distribution of successful anchor recovery under component ablations (RQ3).